DETAILED SUMMARY — Cat vs Dog Classifier using VGG16 Transfer Learning

PROJECT OVERVIEW

This project builds a binary image classification system that can distinguish between cat and dog images with high accuracy (~90-97%). Instead of training a Convolutional Neural Network (CNN) from scratch, we use Transfer Learning with VGG16 — a deep CNN pre-trained on the ImageNet dataset containing 1.2 million images across 1000 categories. The VGG16 convolutional layers are used as a feature extractor, and custom fully connected layers are added on top for the final cat/dog classification decision.

The entire pipeline is designed to run on Google Colab with a T4 GPU runtime and requires no Kaggle account or manual dataset download.

STEP-BY-STEP CODE BREAKDOWN

STEP 1 — Install & Import Dependencies

WHAT IT DOES:

Installs all required Python libraries and imports them. Sets global random seeds to ensure reproducible results across runs. Defines project-wide constants used throughout the pipeline.

KEY LIBRARIES USED:

- TensorFlow / Keras → Deep learning framework for building and training
- tensorflow_datasets → Access to pre-packaged, ready-to-use datasets
- NumPy → Numerical operations and array manipulation
- Matplotlib → Plotting training curves and visualising images
- Scikit-learn → Generating classification reports and confusion matrix
- Seaborn → Styled heatmap for the confusion matrix

CONSTANTS DEFINED:

- IMG_SIZE = (150, 150) → All images resized to 150x150 pixels
- BATCH_SIZE = 32 → Number of images processed per training step
- EPOCHS = 20 → Maximum training rounds (EarlyStopping may stop earlier)
- NUM_CLASSES = 2 → Binary classification: cat (0) or dog (1)
- SEED = 42 → Ensures reproducibility across runs

WHY IT MATTERS:

Setting seeds ensures that shuffling, weight initialization, and dropout produce the same results every time the code is run, making experiments reproducible and comparable.

STEP 2 — Download the Dataset

WHAT IT DOES:

Downloads the "Cats vs Dogs" dataset automatically using TensorFlow Datasets (TFDS). The dataset is split into three subsets: training, validation, and testing. Labels are returned as integers: 0 = cat, 1 = dog.

DATASET DETAILS:

- Source : Microsoft Cats vs Dogs (via tensorflow_datasets)
- Total images : ~23,262 labeled images
- Training set : 80% (~18,610 images)
- Validation set : 10% (~2,326 images)
- Test set : 10% (~2,326 images)
- Image format : RGB JPEG, variable sizes (resized later to 150x150)
- Download size : ~800 MB (cached after first run)

SPLIT STRATEGY:

train[:80%] → Used to update model weights

train[80%:90%] → Used to monitor performance during training (not for updates)

train[90%:] → Used only at the end to report final unbiased accuracy

WHY IT MATTERS:

Keeping test data completely separate from training ensures that the final accuracy score is an honest measure of how the model performs on unseen data.

STEP 3 — Preprocess & Augment the Data

WHAT IT DOES:

Builds efficient tf.data pipelines that load, resize, preprocess, and optionally augment images in parallel before feeding them to the model.

A) PREPROCESSING FUNCTION — resize_and_preprocess():

Applied to ALL splits (train, validation, test).

- Resizes every image to exactly 150x150 pixels
- Converts pixel values to float32
- Applies VGG16's specific preprocessing: subtracts the ImageNet mean RGB values [103.939, 116.779, 123.68] from each channel
- This normalization is critical — VGG16 was trained with these exact pixel statistics, so inputs must match during inference

B) AUGMENTATION FUNCTION — augment():

Applied ONLY to the training set to artificially expand dataset variety.

- Random horizontal flip → Mirrors image left-right (50% chance)
- Random brightness shift → Lightens or darkens the image (±20%)
- Random contrast adjustment → Increases or decreases contrast (±20%)

- Random saturation change → Makes colors more vivid or muted (±20%)
- Random zoom/crop → Crops between 75-100% then resizes back (simulates camera zoom effect)

C) tf.data PIPELINE OPTIMIZATIONS:

- .map() → Applies functions in parallel using all CPU cores (AUTOTUNE)
- .shuffle() → Randomizes order every epoch to prevent ordering bias
- .batch() → Groups 32 images together for efficient GPU utilization
- .prefetch()→ Loads the next batch while the GPU is training on the current one — eliminates idle GPU time

WHY AUGMENTATION MATTERS:

With ~18,000 training images, augmentation effectively multiplies the dataset variety, preventing the model from memorizing exact training images (overfitting) and making it robust to real-world photo variations like different lighting, angles, and zoom levels.

STEP 3a — Visualise Sample Training Images

WHAT IT DOES:

Displays a 2x6 grid of 12 sample images from the raw (un-preprocessed) training dataset, each labeled with its class name (cat/dog). This step is a sanity check to confirm the dataset was loaded correctly before training.

STEP 4 — Build the Model

WHAT IT DOES:

Constructs the complete neural network by combining a frozen VGG16 base with a custom classification head using the Keras Functional API.

A) VGG16 BASE MODEL:

- Loaded with weights='imagenet' → Uses weights from ImageNet pre-training
- include_top=False → Removes VGG16's original 1000-class classifier (we add our own)
- input_shape=(150, 150, 3) → Accepts our 150x150 RGB images
- base_model.trainable = False → Freezes ALL 13 conv layers so their weights are NOT updated during training

VGG16 ARCHITECTURE (frozen part):

Block 1: 2x Conv2D(64) + MaxPooling

Block 2: 2x Conv2D(128) + MaxPooling

Block 3: 3x Conv2D(256) + MaxPooling

Block 4: 3x Conv2D(512) + MaxPooling

Block 5: 3x Conv2D(512) + MaxPooling

Output feature map: (4, 4, 512) for 150x150 input

B) CUSTOM CLASSIFICATION HEAD:

Added on top of VGG16's output feature maps.

Layer 1 — GlobalAveragePooling2D:

Converts the (4, 4, 512) feature map to a flat vector of 512 values.

Preferred over Flatten because it dramatically reduces parameters

(512 vs 8192) and has built-in regularization effect.

Layer 2 — Dense(256, activation='relu'):

256 neurons with ReLU activation. Learns high-level combinations of

VGG16's extracted features specific to cats and dogs.

Layer 3 — Dropout(0.5):

Randomly deactivates 50% of neurons during each training step.

Forces the network to learn redundant representations, strongly

preventing overfitting.

Layer 4 — Dense(128, activation='relu'):

128 neurons for further feature refinement and compression.

Layer 5 — Dropout(0.3):

Randomly deactivates 30% of neurons. Lighter than the first dropout

since the network is deeper and features are more abstract.

Layer 6 — Dense(2, activation='softmax'):

Output layer with 2 neurons — one per class.

Softmax converts raw scores to probabilities that sum to 1.0:

[0.92, 0.08] → 92% cat, 8% dog → Predicted: CAT

C) MODEL COMPILATION:

- Optimizer : SGD (Stochastic Gradient Descent)

  learning_rate=0.001, momentum=0.9

  Momentum helps the optimizer move faster through flat loss regions and avoid getting stuck in local minima.

- Loss : sparse_categorical_crossentropy

  Used because labels are integers (0, 1), not one-hot encoded.

  Measures the difference between predicted probabilities and true labels; the model minimizes this during training.

- Metric : accuracy

  Percentage of correctly classified images.

FULL MODEL ARCHITECTURE SUMMARY:

Input (150x150x3)

└► [VGG16 Block1-5: 13 Conv layers, FROZEN] → output (4x4x512)

└► GlobalAveragePooling2D → (512,)

└► Dense(256, ReLU) → (256,)

└► Dropout(0.5)

└► Dense(128, ReLU) → (128,)

└► Dropout(0.3)

└► Dense(2, Softmax) → [cat_prob, dog_prob]

TRAINABLE PARAMETERS: ~133,000 (only the custom head)

FROZEN PARAMETERS : ~14,714,688 (VGG16 conv layers)

TOTAL PARAMETERS : ~14,847,688

STEP 5 — Train the Model

WHAT IT DOES:

Trains the custom classification head using the prepared data pipelines, guided by three intelligent callbacks that prevent overfitting and wasted computation.

CALLBACKS EXPLAINED:

1. EarlyStopping (monitor='val_accuracy', patience=4):
   - Monitors validation accuracy after every epoch
   - If it does not improve for 4 consecutive epochs, training stops
   - restore_best_weights=True ensures the model reverts to the best checkpoint seen during training, not the last epoch's weights
   - Prevents wasting time training epochs that are no longer improving

2. ReduceLROnPlateau (monitor='val_loss', factor=0.5, patience=2):
   - Monitors validation loss after every epoch
   - If loss does not improve for 2 consecutive epochs, it cuts the learning rate in half (multiplies by factor=0.5)
   - Minimum LR is capped at 1e-6 to avoid zero-step updates
   - Allows fine-grained weight updates when the model is near convergence

3. ModelCheckpoint (monitor='val_accuracy', save_best_only=True):
   - Saves the model weights to 'best_model.keras' only when val_accuracy improves above its previous best

- Guarantees that the best-ever weights are preserved even if subsequent epochs overfit

TRAINING PROCESS:

- Each epoch processes all 18,610 training images in batches of 32
- After each epoch, the model is evaluated on 2,326 validation images
- Weights are updated using backpropagation through only the custom head (VGG16 layers are frozen and receive no gradient updates)
- Expected training time: 3-8 minutes on a T4 GPU

STEP 6 — Plot Training History

WHAT IT DOES:

Generates a side-by-side plot of training metrics across all epochs using the plot_history() function.

Left plot — Model Accuracy:

Blue line = Training accuracy per epoch

Red line = Validation accuracy per epoch

Healthy training: both lines rise together and converge closely

Right plot — Model Loss:

Blue line = Training loss per epoch

Red line = Validation loss per epoch

Healthy training: both lines decrease and converge closely

WHAT TO LOOK FOR:

- If training accuracy is much higher than validation accuracy → Overfitting
- If both are low and close together → Underfitting
- If validation accuracy plateaus while training rises → Overfitting
- Ideal: both curves close together and improving → Good fit

STEP 7 — Evaluate on the Test Set

WHAT IT DOES:

Loads the best saved weights and evaluates the model on the held-out test set (data the model has NEVER seen during training or validation).

Produces three outputs:

A) Basic Metrics:

- Test Accuracy → Overall percentage of correct predictions
- Test Loss → Cross-entropy loss value on test data

B) Classification Report (per-class breakdown):

For each class (cat, dog):

- Precision → Of all images predicted as this class, what % were correct?
- Recall → Of all actual images of this class, what % were found?
- F1-Score → Harmonic mean of precision and recall (balance metric)
- Support → Number of actual test samples for this class

C) Confusion Matrix (2x2 heatmap):

Rows = True labels | Columns = Predicted labels

	Predicted Cat	Predicted Dog
Actual Cat	True Pos	False Neg
Actual Dog	False Pos	True Pos

Dark blue diagonal = correct predictions

Off-diagonal cells = misclassifications

EXPECTED RESULTS:

- Accuracy : 90-95% (frozen base)
- Precision : ~0.91-0.95 for both classes
- Recall : ~0.91-0.95 for both classes
- F1-Score : ~0.91-0.95 for both classes

STEP 8 — Predict on New Images

WHAT IT DOES:

Provides two functions for classifying brand-new images not in the dataset.

A) predict_image(img_path) FUNCTION:

Processing pipeline:

1. Loads the image from disk and resizes to 150x150
2. Converts to NumPy array and adds batch dimension: (1, 150, 150, 3)
3. Applies VGG16 preprocess_input (same normalization as training)

4. Runs model.predict() to get class probabilities

5. Takes argmax to find the predicted class index

6. Returns a dictionary with:

- predicted_class : 'cat' or 'dog'

- confidence : percentage score for the predicted class

- all_probabilities : scores for both classes

Visual outputs:

- Displays the image with the prediction and confidence overlaid

(blue title = cat, orange title = dog)

- Shows a horizontal bar chart with both class probabilities side-by-side

B) predict_from_url(url) FUNCTION:

- Downloads the image from any public URL to a local temp file

- Calls predict_image() on the downloaded file

C) Colab File Upload (Option A):

- Uses google.colab.files.upload() to open a local file picker

- The uploaded file is immediately passed to predict_image()

EXAMPLE OUTPUT:

```
{
 'predicted_class'   : 'dog',
 'confidence'        : 97.3,
 'all_probabilities' : {'cat': 2.7, 'dog': 97.3}
}
```

STEP 9 (OPTIONAL) — Fine-Tune VGG16 Block 5

WHAT IT DOES:

After the custom head has fully converged, this optional step unlocks the last convolutional block of VGG16 (block5) and fine-tunes it alongside the custom head using a very small learning rate.

WHY ONLY BLOCK 5:

- Earlier blocks (block1-3) learn very generic features (edges, textures, colors) that are universally useful — no need to change them
- Block 4-5 learn more dataset-specific features — for ImageNet categories that are far from cats/dogs, these can be slightly suboptimal
- Fine-tuning only block5 gives the best accuracy gain with minimal risk of destroying the learned generic features

FINE-TUNING DETAILS:

- Learning rate reduced to 1e-4 (10x smaller than initial training)
- Very small LR is critical — too large would overwrite the pre-trained weights and cause catastrophic forgetting
- Runs for up to 10 more epochs with its own EarlyStopping and ReduceLROnPlateau callbacks
- Best weights saved separately as 'best_finetuned_model.keras'

EXPECTED ACCURACY GAIN:

- Before fine-tuning : ~90-95%
- After fine-tuning : ~95-97%

STEP 10 — Save & Export the Final Model

WHAT IT DOES:

Saves the trained model in two formats and downloads it to the local machine.

A) Keras Native Format (.keras):

- model.save('cat_dog_vgg16_final.keras')
- Single file containing architecture + weights + optimizer state
- Can be reloaded with tf.keras.models.load_model()
- Best format for continued training or Python-based inference

B) TensorFlow SavedModel Format (directory):

- model.export('cat_dog_savedmodel/')
- Standard format for deployment via:
  → TensorFlow Serving (REST/gRPC inference API)
  → TensorFlow Lite (mobile/edge deployment)
  → TensorFlow.js (browser-based inference)

C) Download:

- files.download() triggers a browser download of the .keras file
  so it is saved to the user's local machine from the Colab environment

KEY DESIGN DECISIONS & RATIONALE

1. WHY VGG16?

VGG16 is well-established, simple in architecture, and extremely effective for transfer learning on natural images. Its 13 convolutional layers have learned rich feature representations from 1.2 million ImageNet images.

2. WHY FREEZE THE BASE?

The dataset (~18,000 images) is small relative to ImageNet (1.2M). Training all layers from scratch would cause severe overfitting. Freezing the base retains powerful generic features while only learning the cat/dog-specific classification mapping.

3. WHY GlobalAveragePooling2D INSTEAD OF Flatten?

Flatten would produce 4*4*512 = 8,192 features per image, leading to ~2 million parameters in the first FC layer. GAP reduces this to 512, dramatically lowering overfitting risk and training time.

4. WHY SGD WITH MOMENTUM INSTEAD OF ADAM?

SGD with momentum often generalizes better than adaptive optimizers like Adam on transfer learning tasks, especially when fine-tuning. It is also the optimizer specified in the assignment requirements.

5. WHY sparse_categorical_crossentropy INSTEAD OF categorical_crossentropy?

Our labels are integers (0 and 1), not one-hot vectors ([1,0] and [0,1]).

sparse_categorical_crossentropy handles integer labels directly, avoiding the need for manual one-hot encoding.

6. WHY DROPOUT IN TWO LAYERS?

Dropout(0.5) after the first FC layer provides strong regularization where the model has the most parameters. Dropout(0.3) after the second provides lighter regularization where features are more refined. Together they significantly reduce overfitting.

7. WHY tf.data INSTEAD OF ImageDataGenerator?

tf.data pipelines are faster, more flexible, and production-grade.
They support parallel loading, prefetching, and seamless GPU pipeline integration. ImageDataGenerator is older and runs on the CPU only.

EXPECTED PERFORMANCE SUMMARY

Phase	Expected Accuracy
After initial training	90 - 95%
After fine-tuning	95 - 97%
Training time (T4 GPU)	~3 - 8 minutes (initial) + ~2 - 4 min (fine-tune)
Inference time	< 50 ms per image